CSE485/CSE488 Deep Learning
Assignment 1: MicroGrad (Scalar Autograd Engine)

Saifeldin Tamer Mohamed ElSayes 22P0233

November 3, 2026

1 Overview

We implement a scalar reverse-mode automatic differentiation engine (`src/engine.py`) and a small neural-network API (`src/nn.py`) on top of it. Every arithmetic operation creates a new `Value` node that records its inputs (`_prev`) and a closure `_backward` that applies the local derivative via the chain rule. All 12 autograder tests pass.

2 Mathematical Derivations

Let L be the final output and $\bar{x} = \partial L/\partial x$ denote the gradient stored in `x.grad`. For an output node z with upstream gradient $\bar{z}$, each operation contributes the following to its inputs:

Operation	Forward	Backward (accumulated with `+=`)
Addition	$z = a + b$	$\bar{a}$ += $\bar{z}$, $\bar{b}$ += $\bar{z}$
Multiplication	$z = ab$	$\bar{a}$ += $b\,\bar{z}$, $\bar{b}$ += $a\,\bar{z}$
Power (n constant)	$z = a^n$	$\bar{a}$ += $na^{n-1}\,\bar{z}$
Tanh	$z = \tanh a$	$\bar{a}$ += $(1 - z^2)\,\bar{z}$
Exp	$z = e^a$	$\bar{a}$ += $z\,\bar{z}$

Tanh. $\tanh a = \frac{e^a - e^{-a}}{e^a + e^{-a}}$; by the quotient rule $\frac{d}{da}\tanh a = \frac{(e^a+e^{-a})^2 - (e^a - e^{-a})^2}{(e^a+e^{-a})^2} = 1 - \tanh^2 a$, so the backward pass reuses the forward output z.

Derived operations. Negation, subtraction and division are composed from the primitives: $-a = a \cdot (-1)$, $a - b = a + (-b)$, $a/b = a \cdot b^{-1}$. Their gradients follow automatically, e.g. $\partial(a/b)/\partial b = -ab^{-2}$. Division by zero raises `ZeroDivisionError` from 0^{-1} in the forward pass.

Neuron. $y = \tanh(\sum_i w_i x_i + b)$, so $\partial y/\partial w_i = (1 - y^2)\,x_i$ and $\partial y/\partial b = 1 - y^2$.

3 Design Justifications

- **Gradient accumulation (`+=`).** A node used in several places (e.g. $c = a + a$) receives gradient from every path (multivariate chain rule). Assigning with `=` would overwrite earlier contributions.

- **Topological sort (post-order DFS).** A node is appended only after all of its children, so iterating the list in reverse guarantees that a node's gradient is complete before it is propagated further. The `visited` set prevents revisiting shared nodes (diamond graphs), which would otherwise cause double counting.
- **Type safety.** Operands are wrapped only if they are `Value`, `int` or `float`; anything else raises `TypeError`. `__pow__` asserts a constant exponent, since $\partial(a^b)/\partial b$ is not supported.
- **Closures for `_backward`.** Each closure captures exactly the values it needs (inputs and output), keeping the engine compact.
- **NN API.** `Neuron`, `Layer`, `MLP` mirror PyTorch: weights are drawn from $U(-1, 1)$, `parameters()` returns a flat list and `zero_grad()` resets gradients before each backward pass. A layer with one neuron returns a scalar `Value` for convenience.

4 Experiment

The demo notebook (`demo.ipynb`) verifies one gradient against finite differences (autograd 0.138672 vs. numerical 0.138672), then trains an MLP $2 \to 16 \to 16 \to 1$ (337 parameters) on a 60-point two-moons dataset using MSE loss and SGD with a decaying learning rate. After 60 steps the loss falls from 1.10 to 0.14 and training accuracy reaches 96.7%.